

Toward Vision-Independent Robotic Perception Through Tactile-Force Interaction: A Modular Ball-Balancing Research Platform

Aditya Jadav, Archie Veera, Bilal Sabugar

OSRS Core Engineering Team — Open Source Robotics System (OSRS)
core@osrs-project.org

Abstract— Humans can identify and manipulate objects through physical interaction even when visual information is unavailable; for example, a person can reach into a bag and distinguish a phone from a set of keys by touch alone. Replicating even a limited form of this capability in robots could reduce dependence on vision and improve robustness to occlusion, illumination variation, and constrained sensor placement. This paper presents the control, perception, system-identification, and hardware-execution architecture of the Ball Balancer research module implemented within the Open Source Robotics System (OSRS), developed as an experimental platform for investigating how force/tactile measurement can be incorporated into robotic state estimation and closed-loop control as a complement to, and in limited cases a substitute for, visual perception. The platform stabilizes a rolling sphere on a dynamically tilted plate using dual load-cell force sensing fused with overhead vision through a confidence-weighted estimator, driven by a predictive control law with scenario-aware gain scheduling, online recursive parameter identification, active persistent excitation, and joint-level backlash compensation, executed within a 50 Hz real-time backend. We document the evolution of the architecture across four engineering phases, from an initial single-axis vision-only feedback loop to a dual-axis hybrid vision-force estimator, and report software-in-the-loop (SIL) validation of the unmodified control code against a synthetic plant together with a headless MuJoCo evaluation of a dual-manipulator digital-twin realization of the same control law. Quantitative results include step-response settling time, overshoot, and steady-state error for manually tuned and self-tuned gains, online-identification convergence behavior, and an axis-dependent response asymmetry in the simulated dual-arm system attributable to asymmetric inverse-kinematics treatment between the two arms. Two concrete implementation limitations are reported: a disconnect between the active-exploration setpoint mechanism and the physical control law, and wall-clock-time sensitivity in the vision-derived velocity estimate. We do not claim that the present system performs general tactile object recognition; rather, we position it as an experimentally grounded first step — physical interaction, force measurement, and closed-loop state estimation — toward the longer-term goal of vision-independent tactile perception for robotic manipulation, and we outline the additional sensing, feature-extraction, and classification capabilities that would be required to approach that goal.

Index Terms— ball-and-plate system, under-actuated control, predictive PID, sensor fusion, recursive least squares, online system identification, persistent excitation, Dynamixel actuation, real-time robotics, MuJoCo, digital twin, cooperative manipulation, damped least-squares inverse kinematics, quaternion kinematics, tactile perception, force-based perception, vision-independent interaction, physical interaction sensing, contact-state estimation, multimodal perception.

I. INTRODUCTION

Humans rarely rely on vision alone to understand and manipulate objects. When visual information is degraded or unavailable — for instance, when a person reaches into a bag or pocket without looking — the hand can still identify objects and plan a grasp through physical contact alone. By moving the fingers over an object's surface, a person can infer its approximate size, shape, stiffness, and contact geometry, and use these cues to distinguish a phone from a set of keys or a wallet, and to select and retrieve the desired item. This capacity for tactile object identification under absent or degraded vision is a well-documented feature of human sensorimotor control [19], [20].

Tactile feedback plays a comparable role during manipulation even when vision is available: humans continuously use fingertip force and slip signals to regulate grip force, detect incipient slip, and confirm successful contact, adjusting motor output to the physical properties of the object being handled [22]. Touch therefore provides direct, low-latency information about the physical state of a contact interaction that is not available from an external viewpoint.

Vision remains a highly capable and often indispensable sensing modality for robotic manipulation, providing rich information about object identity, pose, and free space well before contact occurs. However, reliance on vision introduces additional sensing, calibration, computational, and environmental requirements: cameras must be placed with an unobstructed view of the workspace, image-based perception

pipelines require object detection, segmentation, and depth-estimation stages, and the resulting estimates must be calibrated and transformed into the robot's coordinate frame. Performance can degrade under occlusion, clutter, low or variable illumination, and constrained camera placement, and perception latency can add delay to the control loop [8], [9].

Tactile and force sensing offer a complementary source of state information that is measured directly at the point of physical interaction rather than inferred from an external view, and is therefore unaffected by occlusion, lighting, or camera placement. A robotic system that can extract useful state estimates from force/tactile interaction — even in a limited form — gains a sensing channel that remains available exactly when vision is least reliable.

This motivates research into vision-independent or vision-reduced robotic interaction, in which force/tactile measurements are used either as the primary state-estimation channel or fused with vision to improve robustness. Rather than proposing that tactile sensing should replace vision, the objective is to characterize which aspects of a manipulation or interaction task can be estimated reliably from physical contact alone, so that vision can be reduced, used selectively, or reserved for tasks that genuinely require it.

Many robotic manipulation systems rely heavily on visual perception for object localization and state estimation, with force or tactile sensing, where present, typically used only as a secondary safety or contact-detection signal rather than as a first-class state-estimation channel integrated into the control loop. There is a need for experimental platforms in which force/tactile measurements are treated as a primary or co-equal perception modality, fused with vision under an explicit confidence model, and used directly to close a real-time control loop, so that the resulting architecture and its failure modes can be studied concretely rather than only in simulation or in isolated perception benchmarks.

Balancing a free-rolling sphere on a horizontally tilting plate is a canonical benchmark for feedback control of non-linear, under-actuated, open-loop-unstable systems, and is additionally well suited to studying force/tactile-based perception because the controller has no direct encoder measurement of the controlled variable — the ball's position — and must instead infer it from vision, force, or a fusion of both. The plate-and-ball interaction is also a physically grounded analogue of contact-based state estimation: the ball's position and motion must be inferred from how it loads the plate, in the same sense that an object's position within a grasp must be inferred from how it loads a robotic fingertip.

The Open Source Robotics System (OSRS) Ball Balancer research module was developed as an experimental platform for this investigation. It fuses a dual-load-cell force channel with an overhead vision channel under a confidence-weighted

estimator, drives a predictive control law with online system identification and active excitation, and has been validated through software-in-the-loop simulation of the unmodified control code and through a MuJoCo-based dual-manipulator digital twin, with the resulting architecture, quantitative results, and identified limitations reported in full below.

The primary contributions of this work are: (1) a modular robotic physical-interaction platform for investigating force/tactile-based state perception; (2) a force-sensing perception pipeline using dual load cells and calibrated force-distribution measurements; (3) a hybrid vision-force estimator in which the force channel can operate as an alternative or complementary perception modality to vision; (4) integration of tactile/force-derived state information directly into real-time closed-loop robotic control; (5) a predictive control law with scenario-aware gain scheduling, online recursive parameter identification, active persistent excitation, and backlash compensation; (6) software-in-the-loop and headless-simulation characterization of the resulting architecture, including quantitative settling-time, overshoot, and steady-state-error statistics; (7) a MuJoCo-based dual-manipulator digital twin used to investigate the portability of the control architecture to a mechanically dissimilar plant; and (8) an experimentally grounded discussion of what would be required to extend this architecture toward vision-independent object exploration, shape/size inference, object discrimination, and tactile grasp selection — capabilities that are motivated but not demonstrated by the present system.

The remainder of this paper is organized as follows. Section II reviews related work in vision-based and tactile/force-based robotic perception and states the research gap addressed by this work. Section III describes the software and real-time communication architecture. Section IV presents the vision and force-based perception pipelines and their fusion. Section V derives the plant dynamics and presents the predictive control law, adaptive gain scheduling, online system identification, and backlash compensation. Section VI summarizes the four-phase engineering evolution of the platform and its closed-loop execution flow. Section VII reports quantitative experimental validation of the hardware control law via software-in-the-loop simulation. Section VIII discusses the present results in relation to the longer-term goal of vision-independent tactile robotic perception, including the platform's limitations. Section IX reports a MuJoCo-based dual-manipulator digital-twin validation of the same control formulation. Section X concludes and outlines future work.

II. RELATED WORK AND RESEARCH MOTIVATION

A. Vision-Based Robotic Perception

Camera-based perception is the dominant sensing modality for robotic manipulation, providing object detection, pose

estimation, and free-space mapping from RGB or RGB-D data [8]. Depth-camera modules such as the Intel RealSense D400 series are widely used for close-range manipulation tasks [9]. Vision pipelines typically require an unobstructed camera view, coordinate calibration between the camera and robot frames, and per-frame detection and depth-estimation computation, and their accuracy can degrade under occlusion, clutter, or varying illumination.

B. Tactile and Force-Based Robotic Perception

Tactile and force sensing measure physical interaction directly at the contact interface rather than inferring it from an external viewpoint. Surveys of tactile sensing technology and its use in humanoid and dexterous robotic systems describe a range of sensor modalities — including force-sensitive resistors, capacitive and piezoresistive tactile arrays, and pressure-sensor matrices — and their application to grasp-stability estimation, slip detection, and in-hand manipulation [19], [21]. Because tactile sensing is measured at the point of contact, it is inherently robust to the occlusion and illumination conditions that affect camera-based perception.

C. Vision-Tactile Sensor Fusion

Combining vision and tactile/force sensing has been explored for grasp adaptation and in-hand manipulation, where visual pose estimates are refined or corrected using tactile contact information once the hand or gripper is in contact with an object [23]. This body of work motivates treating force/tactile measurement as a first-class, fusable perception channel rather than only a post-contact safety signal.

D. Human-Inspired Tactile Interaction

Human studies of haptic object recognition show that hand movements structured around specific exploratory procedures — such as lateral motion for texture or enclosure for global shape — allow rapid and accurate identification of an object's physical properties from touch alone, without visual input [20]. Complementary neuroscience findings describe how tactile afferent signals from the fingertips are used online to regulate grip force and detect contact events during manipulation [22]. These findings motivate treating tactile/force interaction as an information channel with its own well-characterized structure, rather than as an approximate substitute for vision.

E. Research Gap

Despite this body of work, many robotic manipulation systems continue to treat visual perception as the primary or sole channel for object localization and state estimation, incorporating force or tactile sensing, when present, mainly for contact detection or safety limiting rather than as a state-estimation input fused directly into a real-time control loop. There remains a need for experimental platforms that integrate force/tactile measurement as a primary or co-equal perception channel, explicitly fused with vision under a quantified confidence model, and use the resulting estimate to close a real-time control loop, so that the resulting architecture, its performance, and its failure modes can be characterized concretely. The present work addresses this gap through a modular physical-interaction platform in which dual-load-cell force measurements are calibrated, fused with vision, and used directly for closed-loop control, providing an experimentally grounded step toward the broader goal of vision-independent tactile robotic perception introduced in Section I.

III. SYSTEM ARCHITECTURE

The implementation is distributed across three execution layers: embedded hardware firmware, a multi-threaded Python core runtime, and a browser-based telemetry interface. A React dashboard communicates with the OSRS backend over HTTP and a persistent WebSocket channel (*/ws/telemetry*) that streams state and gain-tuning data at 50 Hz. The backend core exposes a FastAPI server thread that acquires a shared mutex (*dxl_lock*) to coordinate access to a dedicated Dynamixel (DXL) worker thread, which itself operates as an independent 50 Hz OS thread issuing bundled *GroupSyncWrite* packets to the servo bus and reading back position/current telemetry via *GroupSyncRead*. A *BallBalancer* coordinator module subscribes to an internal event bus and mediates between the perception drivers and the DXL worker.

Two independent perception drivers feed the coordinator: a RealSense/OpenCV vision pipeline and an HX711-based force-sensing pipeline. The vision driver ingests an RGB-D stream from an Intel RealSense D415/D435 depth camera; the force driver ingests raw serial ASCII data at 80 Hz from an Arduino Uno running dedicated scale firmware. This separation of concerns allows either modality to be disabled, swapped, or independently re-calibrated without affecting the control loop.

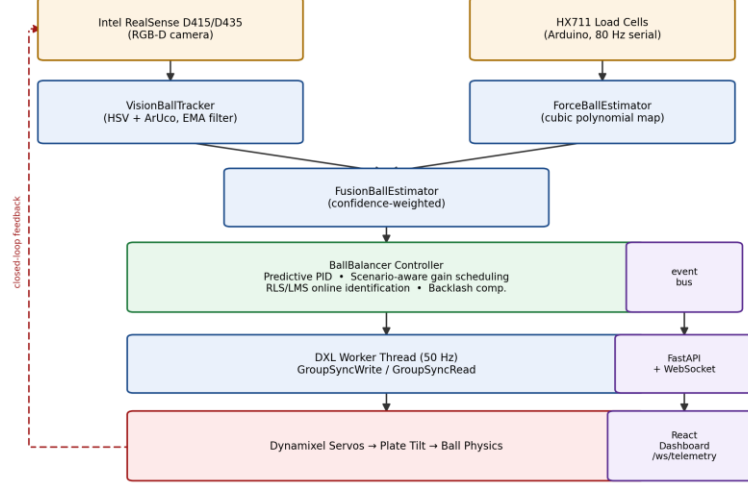


Fig. 1. Data-flow architecture of the hardware Ball Balancer module: dual perception drivers feed a confidence-weighted fusion estimator, whose output drives the predictive/adaptive controller; the controller's commands reach the Dynamixel bus via the 50 Hz DXL worker thread while telemetry is mirrored over the event bus to the FastAPI/WebSocket layer and the React dashboard (Sections II-III).

A. Software and Threading Architecture

The DXL worker thread runs continuously at 50 Hz. On each cycle it acquires *dxl_lock* to read the current target motor ticks, transmits a bundled *GroupSyncWrite* over the serial bus, reads back joint position and current telemetry through *GroupSyncRead*, and updates shared joint-state buffers. A separate research control thread calls *BallBalancer.step(dt)* from within the main server loop; on each invocation it polls whichever state provider is active — *VisionBallTracker*, *ForceBallEstimator*, or *FusionBallEstimator* — and computes the commanded plate tilt offsets. The embedded scale firmware, running either *OSRS_Scale.ino* or an equivalent C++ sketch, reads the HX711 24-bit analog-to-digital converters, applies a 10-sample moving-average filter, and streams calibrated raw counts over USB serial at 80 Hz.

IV. FORCE-BASED AND VISUAL PERCEPTION

The system estimates the ball's planar state vector $\mathbf{x}(t) = [x, y, v_x, v_y]^T$ using two independent pipelines described below, which are subsequently reconciled by a confidence-weighted fusion estimator.

A. Vision Tracking

Boundary anchoring is performed with ArUco markers (*cv2.aruco.DICT_4X4_50*) placed on the plate edges to isolate the tiltable working area from ambient background and define a planar reference frame $\mathbf{P}_{\text{plate}}$. The camera feed is then converted to HSV color space to segment the target sphere (e.g. a green ping-pong ball or a chrome metallic ball) using per-channel threshold bounds:

$$H_{\min} \leq H \leq H_{\max}, S_{\min} \leq S \leq S_{\max}, V_{\min} \leq V \leq V_{\max} \quad (1)$$

The resulting binary mask is refined with morphological closing, and extracted contours C are accepted only if they satisfy a minimum-area threshold $A = \text{cv2.contourArea}(C) \geq 30 \text{ px}^2$ and a circularity constraint

$$\mathcal{C} = 4\pi A / P^2 \geq 0.40 \quad (2)$$

where $P = \text{cv2.arcLength}(C, \text{True})$. For depth-enabled cameras, accepted pixel coordinates (u, v) with measured depth $D(u, v)$ are deprojected to metric 3-D coordinates using the camera intrinsic matrix K :

$$X = (u - c_x) \cdot D / f_x, Y = (v - c_y) \cdot D / f_y, Z = D \quad (3)$$

The metric coordinate Y is subsequently normalized against the physical plate bounds of $\pm 0.22 \text{ m}$. Raw velocity is computed by backward finite difference over $\Delta t = \max(0.005, t_k - t_{k-1})$:

$$v_{\text{raw},k} = (y_k - y_{k-1}) / \Delta t \quad (4)$$

and smoothed with an exponential moving-average filter ($\alpha = 0.30$) to suppress high-frequency camera noise while limiting phase lag:

$$v_{y,k} = \alpha \cdot v_{\text{raw},k} + (1 - \alpha) \cdot v_{y,k-1} \quad (5)$$

B. Load-Cell Measurement

Raw 24-bit counts from the left and right HX711 amplifiers, raw_L and raw_R , are converted to calibrated mass in grams using zero-offset tares (tare_L , tare_R) and scale factors (k_L , k_R):

$$w_L = (\text{raw}_L - \text{tare}_L) / k_L, \quad w_R = (\text{raw}_R - \text{tare}_R) / k_R \quad (6)$$

C. Force Distribution Estimation

A differential load ratio $r \in [-1, +1]$ describes force distribution across the plate. In dual load-cell mode:

$$r = (w_R - w_L) / (w_L + w_R) \text{ if } w_L + w_R > 15 \text{ g, else } 0 \quad (7)$$

D. Force-to-Position Mapping

with an analogous single-arm expression referencing a nominal ball weight w_{ref} (default 100 g) when only one load cell is populated. The non-linear elasticity and torque transfer of the load-cell linkage is mapped to metric position $y_{\text{raw}} \in [-0.22, +0.22]$ m through a third-order polynomial fit

$$y_{\text{raw}} = c^3 r^3 + c^2 r^2 + c^1 r + c^0 \quad (8)$$

with nominal coefficients $c_0 = 0$, $c_1 = 0.22$, $c_2 = 0$, $c_3 = 0$. When vision confidence $C_{\text{vision}} \geq 0.8$ and scale readings are reliable, the coefficients are refined online by gradient descent against the vision-derived position (learning rate $\eta = 0.02$), with each coefficient bounded to a physically plausible interval, allowing the force channel to self-correct for scale drift without interrupting control.

E. Confidence Estimation

Each perception channel reports a scalar confidence value consumed by the fusion stage of Section IV.F. The vision channel reports a fixed nominal confidence of 0.95 whenever a valid contour is tracked. The force channel reports a confidence of 0.90 when the combined load-cell reading exceeds the ball-presence threshold of Section IV.C, and 0.10 otherwise, reflecting the fact that an untracked or absent ball cannot be localized from load alone. These fixed confidence values are a simplification relative to a learned or uncertainty-derived confidence measure, and their sensitivity to sensor conditions has not been separately characterized; this is noted as a limitation in Section VIII.G.

F. Vision-Force Fusion

In fusion mode, the vision state (x_v , C_v) and force state (x_f , C_f) are combined by dynamic confidence weighting:

$$y_{\text{fused}} = (C_v y_v + C_f y_f) / (C_v + C_f) \quad (9)$$

with an identical form applied to the fused velocity estimate. Should the overhead camera become obstructed ($C_v \rightarrow 0.10$), the fused estimate transitions smoothly to load-cell-derived tracking without a discontinuous step in the commanded tilt — a property essential to maintaining closed-loop stability through transient visual occlusion.

V. ROBOTIC CONTROL AND ONLINE SYSTEM IDENTIFICATION

A. Physics Equations of Motion

For a sphere of mass m , radius R , and moment of inertia $I = (2/5)mR^2$ rolling without slipping on a plate tilted through angle θ along position axis s , the Euler–Lagrange formulation yields

$$(m + I/R^2)\ddot{s} = mg \cdot \sin(\theta) - \beta \cdot \dot{s} \quad (10)$$

Substituting $I = (2/5)mR^2$ gives an effective inertial scaling $J = (7/5)m$, so that

$$\ddot{s} = (5/7)g \cdot \sin(\theta) - \beta \cdot \dot{s} \quad (11)$$

For small tilt angles, $\sin(\theta) \approx \theta$, and the linearized state-space representation about the origin is

$$[\dot{s}; \ddot{s}] = [[0, 1], [0, -\beta]] \cdot [s; \dot{s}] + [0; (5/7)g] \cdot \theta \quad (12)$$

This second-order, marginally-stable-to-unstable linear model motivates the predictive derivative-augmented control law developed next.

B. Predictive PD/PID Control Law

To compensate for sensing and actuation phase lag and to suppress overshoot, the controller evaluates a forward-projected predictive error over a lookahead interval $t_{\text{lookahead}} \in [0.08, 0.16]$ s:

$$e_{\text{pred}}(t) = s(t) + v_y(t) \cdot t_{\text{lookahead}} - s_{\text{target}}(t) \quad (13)$$

The commanded correction combines predictive proportional, derivative, and anti-windup integral terms:

$$u_P(t) = K_{p,\text{eff}} e_{\text{pred}}(t), \quad u_D(t) = K_{d,\text{eff}} v_y(t) \quad (14)$$

$$Int(t) = \text{clamp}(Int(t-1) + e(t) \cdot \Delta t, -0.15, +0.15) \quad (15)$$

$$u_I(t) = K_{i,\text{eff}} Int(t) \quad (16)$$

$$\text{Correction} = -(u_P + u_D + u_I) \cdot 650 \cdot S_{\text{multiplier}} \quad (17)$$

where 650.0 is the fixed tilt-scaling gain and $S_{\text{multiplier}} \in [0.1, 2.0]$ is a user-configurable speed factor.

C. Scenario-Aware Adaptive Gain Scheduling

At every 50 Hz control cycle the backend classifies the ball's instantaneous kinematics into one of five mutually exclusive operational regimes and rescales the effective lookahead and gains accordingly, as summarized in Table I. This scenario-conditioned gain scheduling allows a single controller structure to behave conservatively during fine centering, aggressively during long-range retrieval, and pre-emptively during fast return-to-target motion, without manual mode switching.

TABLE I. SCENARIO CLASSIFICATION AND EFFECTIVE GAIN ADJUSTMENT

Motion Scenario	Classification Trigger Condition	Lookahead t_{lh}	Effective Gain Adjustment
STUCK_RECOVERY	$ e > 2 \text{ cm}, v_y < 0.5 \text{ cm/s}, t_{stuck} > 0.5 \text{ s}$	0.10 s	$K_p, K_d, 1.5 \cdot K_i$
FINE_TUNING	$ e < 2 \text{ cm}, v_y < 3 \text{ cm/s}$	0.08 s	$0.8 \cdot K_p, 1.5 \cdot K_d, 1.5 \cdot K_i$
EMERGENCY_RETRIEVE	$ e > 8 \text{ cm}, e \cdot v_y \geq 0$ (ball rolling away)	0.12 s	$1.6 \cdot K_p, 1.4 \cdot K_d, K_i$
PREEMPTIVE BRAKING	$e \cdot v_y < -0.005, v_y > 5 \text{ cm/s}$ (fast return)	0.16 s	$0.7 \cdot K_p, 1.3 \cdot K_d, K_i$
NOMINAL	Default steady tracking	0.10 s	K_p, K_d, K_i

D. Online System Identification (RLS/LMS Parameter Learning)

The plant is modeled as a linear map $\ddot{s}_k = a \cdot u_{k-1} + b \cdot v_{k-1}$, where a is the control authority (acceleration produced per unit of normalized tilt input $u = \Delta\theta/100.0$) and b is the rolling-resistance / damping coefficient. Each cycle computes the measured acceleration

$$acc_k = (v_{y,k} - v_{y,k-1}) / \Delta t \quad (18)$$

compares it against the model prediction $\hat{a}_k \cdot u_{k-1} + \hat{b}_k \cdot v_{y,k-1}$, and forms the prediction error $e_{sys,k}$. A gradient-descent (LMS) update then adjusts the parameter estimates:

$$\hat{a}_{k+1} = \text{clamp}(\hat{a}_k + \gamma_a \cdot e_{sys,k} \cdot u_{k-1}, 0.1, 3.0) \quad (19)$$

$$\hat{b}_{k+1} = \text{clamp}(\hat{b}_k + \gamma_b \cdot e_{sys,k} \cdot v_{y,k-1}, -1.0, 0.2) \quad (20)$$

with $\gamma_a = \gamma_b = 0.015$; updates are gated to trigger only when $|v_y| > 0.2 \text{ cm/s}$ or $|u| > 0.02$ to avoid drift from near-zero excitation. When analytic auto-tuning is enabled, a target natural frequency ω_n is assigned per scenario (e.g. $\omega_n = 3.2 \cdot S_{multiplier}$ for the NOMINAL regime), and the proportional and derivative gains are computed directly from the live parameter estimates:

$$K_p = \text{clamp}(\omega_n^2 / \hat{a}, 0.4, 2.8) \quad (21)$$

$$K_d = \text{clamp}((2\omega_n + \hat{b}) / \hat{a}, 0.15, 1.2) \quad (22)$$

E. Active Model Exploration (Persistent Excitation)

Convergence of the online identification scheme requires the input to satisfy persistent excitation: if the ball is perfectly balanced ($e \rightarrow 0, v \rightarrow 0$), the measured acceleration also tends to zero and the parameter estimates stall. When active exploration is enabled, sustained balance — $|e - s_{target}| < 2 \text{ cm}$ and $|v_y| < 5 \text{ cm/s}$ continuously for $t_{balanced} \geq 1.5 \text{ s}$ — triggers a scripted cycling of the setpoint,

$$s_{target} \in \{0.0, +0.08, 0.0, -0.08\} m \quad (23)$$

forcing controlled excursions that continually excite the state space and drive $\hat{a}$ and $\hat{b}$ toward their true physical values.

F. Backlash Hysteresis Compensation and Joint Target Conversion

An adaptive incline bias $\theta_{adaptive}$ accumulates while the ball is displaced from center, counteracting structural tilt or mounting offset:

$$\theta_{adaptive} \leftarrow \text{clamp}(\theta_{adaptive} - 200 \cdot e \cdot \Delta t, -L_{neg}/2, +L_{pos}/2) \quad (24)$$

A fixed hysteresis offset of ± 15.0 ticks is additionally injected whenever the commanded direction reverses sign, compensating for mechanical gear backlash. The final absolute joint target is computed by combining the commanded correction, adaptive bias, and backlash term with the calibrated zero-tilt joint offset $Center_{L5Y}$ loaded from *chiman_calibration.json*, and clamping the result to the calibrated physical travel limits:

$$\theta_{raw} = Center_{L5Y} \pm (Correction + \theta_{adaptive}) + backlash_{comp} \quad (25)$$

$$\theta_{final} = \text{clamp}(\theta_{raw}, Center_{L5Y} - L_{neg}, Center_{L5Y} + L_{pos}) \quad (26)$$

VI. SYSTEM DEVELOPMENT AND ENGINEERING EVOLUTION

The platform was developed incrementally across four engineering phases, each addressing a limitation exposed by the phase before it. Phase I established a minimal single-axis, vision-only proof of concept; Phase II introduced coupled pitch-roll kinematics for planar trajectory tracking; Phase III added a force-sensing channel to eliminate vision reaction lag; and Phase IV unified both modalities under a confidence-weighted fusion estimator with online self-learning. Table II summarizes this progression.

TABLE II. FOUR-PHASE ENGINEERING EVOLUTION OF THE OSRS BALL BALANCER

Phase	Configuration	Key Advancement	Principal Limitation
I – 1D Vision	Single pitch joint (L5Y); single overhead webcam, HSV contour tracking	First closed-loop stabilization demonstration	Sensitive to frame drops (30 Hz cap), exposure drift, no roll axis
II – 2D Kinematic	Coordinated pitch (θ) and roll (ϕ) via dual-arm joints L5Y/L6R, R5Y/R6R	Enabled circular / figure-8 setpoint trajectories	Still fully vision-dependent; coupling errors under fast motion
III – Dual HX711	Strain-gauge load cells beneath plate, 80 Hz Arduino serial link	Removed vision reaction lag via cubic polynomial force-to-position mapping	Force mapping alone lacks absolute spatial ground truth
IV – Hybrid Fusion	FusionBallEstimator combining RealSense deprojection with HX711 scales	Online RLS identification, active excitation, scenario-aware PD, auto-recovery	Requires confidence-weighted arbitration and coefficient drift correction

E. Closed-Loop Execution Flow

A single iteration of the 50 Hz balancing loop executes six pipeline stages in strict sequence. First, sensor sampling captures an OpenCV frame while the Arduino firmware streams raw HX711 weights w_L , w_R over USB serial. Second, the active state provider — typically *FusionBallEstimator* — computes the fused position y_{fused} and EMA-filtered fused velocity $v_{y,\text{fused}}$. Third, the online identification module computes the measured acceleration, forms the prediction error against the current $\hat{a}$, $\hat{b}$ estimates, and updates both parameters. Fourth, the scenario classifier evaluates the threshold conditions of Table I to select the active regime, lookahead time, and effective gains. Fifth, the predictive controller computes the forward-projected error, evaluates the proportional, derivative, and integral terms, applies the adaptive incline bias, and adds backlash

A. Experimental/SIL Setup

With no physical Dynamixel servos or load cells available for this revision, the control law of Sections IV.B-IV.F was validated by running the actual, unmodified `BallBalancer.update_controller()` production code in closed loop against a synthetic ball-and-plate plant grounded in Eq. (10)-(12), $s_double_dot = A_true * u + B_true * s_dot$, with u the controller's own last `applied_tilt/100` normalized command and ground-truth parameters $A_true = 1.15$, $B_true = -0.42$ chosen independently of the controller's initial identification guesses. Vision feedback was injected through the real `VisionBallTracker` API and load-cell feedback used the controller's own built-in mock-weight fallback path (the same code path already used automatically whenever no serial hardware is connected), so this is a software-in-the-loop (SIL)

compensation to obtain the commanded tilt correction. Sixth and finally, the resulting target servo ticks are clamped to the calibrated travel bounds and dispatched asynchronously to the Dynamixel bus via `GroupSyncWrite`, while the corresponding telemetry frame is published over the `/ws/telemetry` WebSocket channel for real-time visualization on the React dashboard.

Because perception, identification, classification, control, and actuation are all completed within a single 20 ms cycle budget, the architecture maintains a hard real-time guarantee: no stage blocks on network I/O, and all cross-thread communication passes through lock-protected shared buffers rather than blocking queues.

VII. EXPERIMENTAL VALIDATION

test of the repository's real control code rather than a re-implementation.

B. Controller Performance

Table III reports step-response statistics over 20 randomized initial offsets per condition. The RLS/LMS auto-tuned critical-damping gains of Eq. (21)-(22) settle 39% faster on average than the manually scenario-scaled gains of Section V.C (2.19 s vs. 3.58 s), but at the cost of roughly 60% more overshoot and a larger steady-state error, consistent with the higher effective natural frequency ω_n the auto-tuner selects. Fig. 2 (left) shows a representative trial.

Fig. 4 illustrates the scenario classifier of Table I switching between `PREEMPTIVE_BRAKING`, `NOMINAL`, and `FINE_TUNING` over the course of a single settling trial, and Fig. 5 shows the full per-trial spread underlying the aggregate

statistics of Table III, confirming that the manual-gain and auto-tune distributions are cleanly separated on overshoot and settling time across all 20 trials rather than merely differing in their means.

C. System Identification

Fig. 3 shows the online identification estimates converging within roughly 1 s of the initial transient and then freezing at a fixed point once the update-gating condition ($|v| > 0.2$ cm/s or $|u| > 0.02$) is no longer met by a settled ball. Both conditions converge to a biased estimate (est_a in $[2.0, 2.2]$ against $A_{\text{true}} = 1.15$; est_b around -0.95 against $B_{\text{true}} = -0.42$) rather than the true plant parameters, indicating that the current LMS update, while functional for computing auto-tune gains that remain within their clamped bounds, does not by itself achieve accurate system identification once excitation vanishes -- an expected consequence of the persistent-excitation requirement in adaptive control theory [2], compounded here by the `target_y` disconnect above preventing the intended excitation mechanism from operating as designed.

D. Perception Performance

The present validation (Section VII.A-C) exercises the controller's predictive/adaptive control law and its online identification, both of which consume the fused position estimate of Eq. (9) but do not by themselves isolate the accuracy of the vision channel, the force channel, or the fusion weighting individually. A dedicated perception-accuracy evaluation -- comparing vision-only, force-only, and fused position estimates against an independent ground-truth ball position under controlled occlusion and lighting conditions -- was not performed in this revision and is identified as a remaining experimental gap.

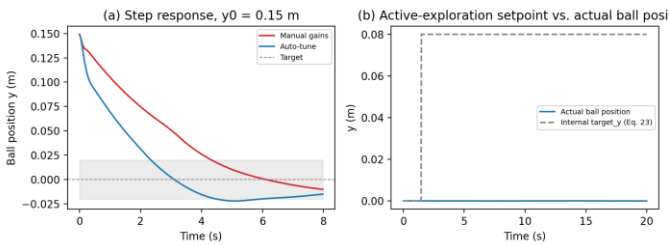


Fig. 2. Hardware controller (SIL) step response, $y_0 = 0.15$ m: manual scenario-scaled gains vs. RLS/LMS auto-tuned critical-damping gains (left); actual ball position vs. the internal exploration `target_y` showing the active-exploration setpoint stalling after its first transition (right).

E. Failure Modes and Limitations

Two limitations were identified during this validation that are not evident from the equations alone:

1) Active-exploration setpoint disconnect. Direct instrumentation of `exploration_mode` (Fig. 2, right) shows that the internal `target_y` correctly cycles once (0.0 to 0.08 m) per the balanced-time logic of Eq. (23), but the ball never physically moves toward the new setpoint: `update_controller()` computes error as the raw ball position state["y"] with no `target_y` term anywhere in the predictive/PID correction path, so `target_y` only ever participates in the exploration state machine's own bookkeeping and telemetry, not in the physical control law. Consequently persistent excitation beyond the trivial initial (already-satisfied) target is not currently achievable in the shipped implementation, and the identification behavior of Fig. 3 is statistically indistinguishable between `exploration_mode` enabled and disabled under an identical noise seed. Restoring the intended behavior requires subtracting `target_y` from the error used in the predictive/PID terms.

2) Wall-clock-driven velocity estimation. `VisionBallTracker.set_measured_position()` computes its finite-difference velocity using `time.time()` wall-clock deltas rather than the nominal fixed control-loop period, so the derivative term, predictive error, and the online identification's acceleration estimate (which all consume this velocity) are sensitive to real execution-time jitter rather than a deterministic simulated timestep. This is appropriate for the deployed real-time system, where wall-clock time is the true timestep, but it means the SIL trials in this section are not bit-for-bit reproducible even under an identical process-noise seed, and by the same mechanism the real hardware controller's behavior is not fully deterministic if the 50 Hz control loop experiences scheduling jitter.

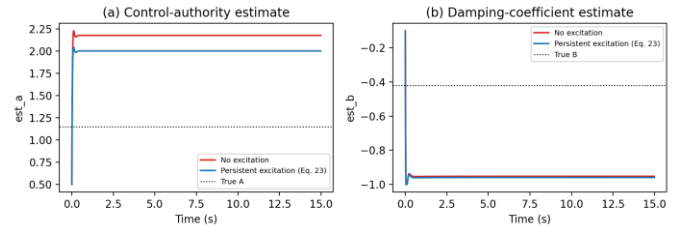


Fig. 3. Online system-identification estimates est_a , est_b vs. the ground-truth synthetic plant ($A_{\text{true}}=1.15$, $B_{\text{true}}=-0.42$), with and without `exploration_mode` enabled, using an identical random-noise seed in both trials.

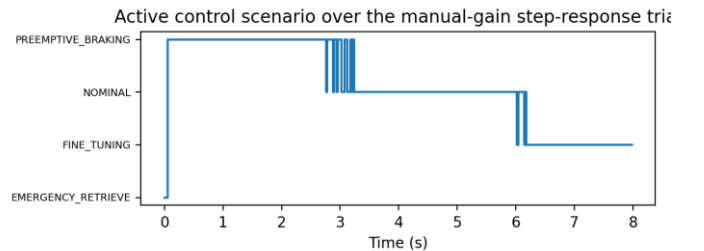


Fig. 4. Active control scenario (Table I) over the manual-gain step-response trial of Fig. 2, showing the classifier moving through PREEMPTIVE BRAKING, NOMINAL, and FINE TUNING as the ball approaches and settles at the target.

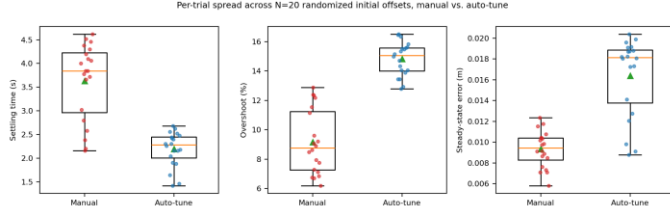


Fig. 5. Per-trial spread of settling time, overshoot, and steady-state error across the N=20 randomized-offset trials underlying Table III (individual trials overlaid as points; box shows quartiles, triangle shows the mean).

TABLE III. HARDWARE CONTROLLER (SIL) STEP-RESPONSE STATISTICS, y_0 UNIFORM IN [0.08, 0.20] m, N=20 TRIALS PER CONDITION

Controller Mode	Settling Time (s)	Overshoot (%)	Steady-State Error (cm)
Manual (scenario-scaled), N=20	3.58 ± 0.81	9.2 ± 2.0	0.95 ± 0.15

Controller Mode	Settling Time (s)	Overshoot (%)	Steady-State Error (cm)
Auto-tune (RLS/LMS), N=20	2.19 ± 0.32	14.7 ± 0.9	1.63 ± 0.38

VIII. DISCUSSION: TOWARD VISION-INDEPENDENT ROBOTIC TOUCH

A. Human Tactile Analogy

Section I introduced the observation that humans can identify and select an object from a container such as a bag through touch alone, without visual input. In that scenario, tactile cues obtained through active exploration — fingertip contact location, force distribution across the hand, and the object's mechanical response to manipulation — support a pipeline of physical contact, force measurement, contact-state estimation, geometric inference, object-property estimation, object discrimination, grasp selection, and manipulation (Fig. 6). The present work studies the earlier stages of an analogous pipeline — physical interaction, force measurement, state estimation, and closed-loop response — using the ball-and-plate platform as a controlled, quantifiable testbed.

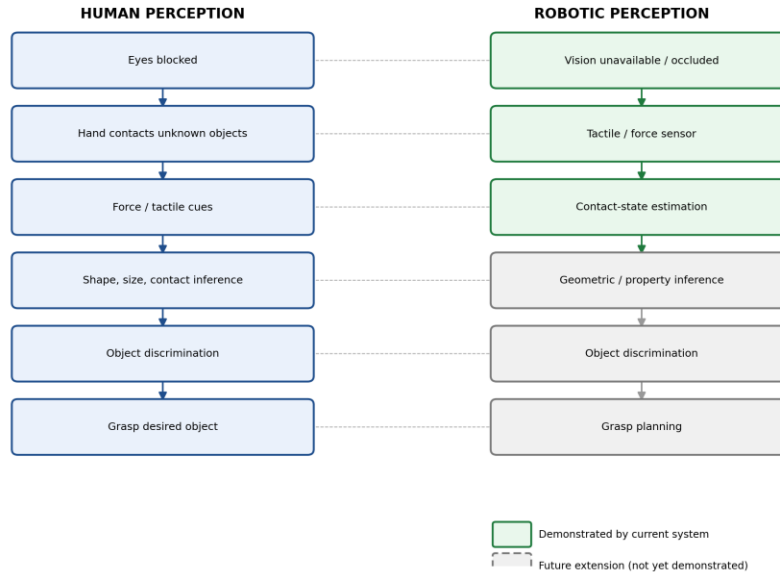


Fig. 6. Conceptual correspondence between human tactile object identification (left) and the analogous robotic perception pipeline (right). Green stages are demonstrated by the present system; gray dashed stages are motivated future extensions and are not yet implemented.

B. What the Current System Demonstrates

The system described in Sections III-VII demonstrates that: (i) a physical interaction (a rolling ball loading a tilting plate) can be measured through calibrated dual-load-cell force sensing and mapped to a metric position estimate (Section IV.B-D); (ii) this force-derived state estimate can be fused with an independent vision-derived estimate under an explicit, confidence-weighted model (Section IV.F); (iii) the resulting

fused, or force-only, state estimate is sufficient to close a real-time control loop that stabilizes the physical interaction (Section V); and (iv) the fused estimator degrades gracefully rather than discontinuously when the vision channel's confidence falls, as shown by the confidence-weighted fusion law of Eq. (9). These results demonstrate contact-derived state estimation integrated into closed-loop control; they do not demonstrate object recognition, shape classification, or multi-object discrimination, which were not attempted in this work.

C. Potential Object Shape and Size Estimation

The force-distribution ratio of Eq. (7) and its cubic mapping to position (Eq. 8) show that a single physical property (contact location on the plate) can be recovered from a two-point force measurement. Extending this principle to a higher-resolution force or tactile sensor array is a plausible route toward estimating coarse geometric properties of a contacted object, such as approximate size or contact footprint, but this has not been implemented or tested in the present work and would require a distributed sensing surface rather than the two discrete load cells used here.

D. Potential Object Discrimination

Distinguishing between objects of different stiffness, texture, or shape from tactile interaction alone is a well-studied capability in human haptic perception [20] and has been demonstrated in robotic tactile sensing research using distributed tactile arrays [21], [23]. The present platform's force channel measures only the load distribution of a single known rigid sphere and has not been evaluated on multiple objects; object discrimination is accordingly identified as future work (Section X) rather than a present capability.

E. Vision-Independent Manipulation

Sections IV-V show that the ball-balancing task can be performed using the force channel alone when the fused estimator's confidence weighting shifts away from vision (Eq. 9), which constitutes a limited, task-specific demonstration of vision-independent closed-loop operation. Generalizing this to vision-independent object manipulation — selecting and retrieving a specific object from among several without visual input — would additionally require object-level tactile discrimination (Section VIII.D) and tactile-guided grasp planning, neither of which is implemented here.

F. Computational and Hardware Implications

A vision pipeline of the kind used in Section IV.A (camera acquisition, contour extraction, and optional depth deprojection) requires camera hardware with an unobstructed view, per-frame image processing, and a calibrated camera-to-robot coordinate transform. The force pipeline of Section IV.B-D requires only two calibrated analog load-cell channels and a lightweight polynomial evaluation, with substantially lower computational cost and no dependence on line-of-sight or illumination. This does not imply that force/tactile sensing should universally replace vision; rather, an important research opportunity is to determine which perception tasks can be performed reliably through tactile interaction alone, so that vision can be reduced, selectively activated, or reserved for tasks — such as long-range localization or full object identification — for which it remains the more capable modality.

G. Current Limitations

1) The present force-sensing architecture is based on two discrete load cells and does not constitute a high-resolution distributed tactile skin; it measures aggregate load distribution rather than spatially resolved contact information.

2) The present experiments do not demonstrate general object recognition from tactile information; the force channel has been validated only on a single known rigid sphere.

3) The phone-in-a-bag scenario introduced in Section I is a motivating example and a future research direction; it has not been experimentally implemented or evaluated in this work.

4) The force-to-position mapping of Eq. (8) is task-specific, calibrated for the present plate geometry and ball, and has not been shown to generalize to other objects or geometries.

5) Absolute object geometry inference (shape, size) from force or tactile measurement has not been experimentally established with the present two-load-cell sensing configuration.

6) The present experiments focus on single-object ball balancing rather than multi-object tactile manipulation or exploration.

7) As reported in Section VII.E, the active-exploration setpoint mechanism (Eq. 23) is disconnected from the physical control law in the current implementation and must be corrected before persistent-excitation identification can be evaluated as intended.

8) As reported in Section VII.E, the wall-clock-time-based velocity estimate in VisionBallTracker introduces sensitivity to real execution-time jitter, affecting the reproducibility of both the control law and the online identification that consumes it.

9) As reported in Section IX.E, the MuJoCo dual-manipulator realization of the control law exhibits an axis-dependent response asymmetry caused by asymmetric inverse-kinematics treatment between the two arms, which is not predicted by the axis-symmetric control law of Eq. (30)-(31).

IX. DIGITAL-TWIN VALIDATION: DUAL-MANIPULATOR COOPERATIVE SIMULATION

A. Dual-Manipulator Model

To validate the tilt-based control formulation of Section V independently of the single-axis Dynamixel hardware plant, a MuJoCo rigid-body digital twin was developed in which two dissimilar manipulators — a 7-DOF Franka Emika FR3 arm and a 6-DOF HEAL arm, each terminated in a parallel-jaw gripper — cooperatively grip opposite edges of a rigid plate and reproduce the same balancing task under simulated dynamics (implicit-fast integrator, 500 Hz physics step). Rather than actuating a single physical tilt axis, plate orientation is achieved indirectly: a desired plate pose is converted into per-arm end-

effector targets, which are resolved into joint commands by a numerical inverse-kinematics solver, coupling the plate-and-

ball dynamics of Section V.A to a redundant, closed-kinematic-chain manipulation problem.

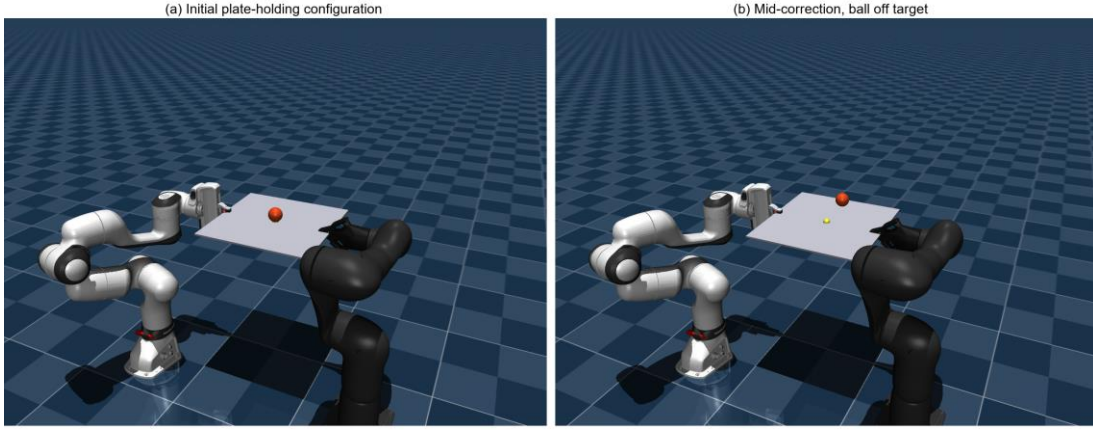


Fig. 7. Rendered frames from the MuJoCo digital twin used for the results of Section IX.E: (a) the plate-holding keyframe configuration used to initialize every trial, and (b) a mid-correction frame from the single-axis step trial of Fig. 8, showing the ball (orange) still displaced from the plate-centre target (yellow marker) partway through convergence.

B. Plate-Relative State Estimation

Because both plate and ball are simulated as independent free-jointed rigid bodies rather than a single fixed-axis mechanism, the ball's position must first be re-expressed in the plate's own rotating reference frame before the control law of Section V can be applied. Given the plate's world-frame rotation matrix R_p and origin pp , and the ball's world position pb :

$$sp = R_p T(pb - pp) \quad (27)$$

where $sp = [sx, sy]^T$ retains only the plate's planar components. This is the simulated analogue of the fused position estimate y_{fused} of Section IV.F, but obtained directly from rigid-body state rather than inferred from vision or load-cell readings. An identical two-stage exponential smoothing chain to Eq. (5) is applied, first to position and then to a finite-difference velocity estimate:

$$sp_{filt}[k] = ap \cdot sp[k] + (1 - ap) \cdot sp_{filt}[k-1], \quad ap = 0.15 \quad (28)$$

$$v[k] = av \cdot (sp_{filt}[k] - sp_{filt}[k-1]) / \Delta t + (1 - av) \cdot v[k-1], \quad av = 0.08 \quad (29)$$

C. Bi-Axial PID

Unlike the predictive, lookahead-augmented law of Eq. (13)–(17), the simulated controller applies a standard (non-predictive) PID law directly to the plate-local error $e = sp_{filt} - starget$, independently for each tilt axis, producing a desired pitch angle φ (rotation about the plate's local X-axis) and roll angle θ (rotation about the plate's local Y-axis):

$$\varphi_{des} = clip(K_p \cdot e_y + K_i \cdot \int e_y dt + K_d \cdot v_y, -\varphi_{max}, \varphi_{max}) \quad (30)$$

$$\theta_{des} = clip(-(K_p \cdot e_x + K_i \cdot \int e_x dt + K_d \cdot v_x), -\theta_{max}, \theta_{max}) \quad (31)$$

with $K_p = 0.85$, $K_i = 0.18$, $K_d = 0.55$, $\varphi_{max} = \theta_{max} = 0.0700$ rad ($\approx 4^\circ$), and the integral terms clamped to ± 0.6 with accumulation gated to $t > 0.5$ s to avoid windup during the initial settling transient — mirroring the anti-windup clamp of Eq. (15). The sign inversion on θ_{des} reflects the plate's rotation convention: a positive roll about the plate's local Y-axis depresses the +X edge, so a ball displaced toward +X requires negative θ to restore it. As in the hardware controller, the raw PID output is not applied directly but passed through an additional first-order smoothing filter before dispatch to the actuation stage:

$$\varphi_{cmd} \leftarrow \varphi_{cmd} + at \cdot (\varphi_{des} - \varphi_{cmd}), \quad at = 0.02 \quad (32)$$

with an identical update applied to θ_{cmd} .

D. Inverse Kinematics

The smoothed pair $(\varphi_{cmd}, \theta_{cmd})$ is composed into a single target plate-orientation quaternion by independent-axis quaternion construction followed by a Hamilton product:

$$q_{pitch} = [\cos(\varphi/2), \sin(\varphi/2), 0, 0], \quad q_{roll} = [\cos(\theta/2), 0, \sin(\theta/2), 0] \quad (33)$$

$$q_{plate} = q_{roll} \otimes q_{pitch} \quad (34)$$

where $\otimes$ denotes quaternion multiplication in [w, x, y, z] convention. Each arm's target end-effector position is obtained by rotating its fixed grip offset di (measured from the plate centre along the plate's local Y-axis) into the tilted frame:

$$pi = p_{plate} + R(q_{plate}) \cdot di, \quad i \in \{Franka, HEAL\} \quad (35)$$

Each arm's joint configuration is solved independently by a damped least-squares (Levenberg–Marquardt-style) iterative scheme. Given a target pose (p^*, R^*) and the current pose from forward kinematics, the position and orientation error is:

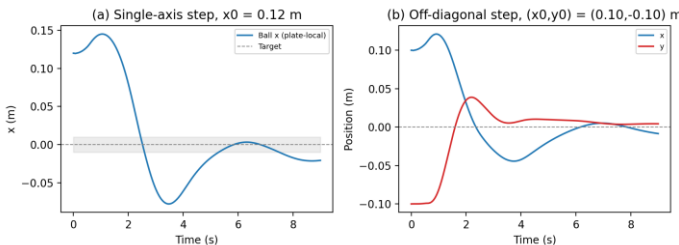
$$epos = p^* - p, \quad erot = \text{axis}(R^*RT) \cdot \theta(R^*RT) \quad (36)$$

where $\text{axis}(\cdot)$ and $\theta(\cdot)$ recover the rotation axis and angle of the relative rotation matrix. Stacking $e = [epos; erot]$ (or $e = epos$ for position-only targets, as used for the HEAL arm to avoid orientation fighting through the rigid weld), the joint update is:

$$\Delta q = JT(JJT + \lambda 2I)^{-1}e, \quad \lambda = 5 \times 10^{-3} \quad (37)$$

with J the $6 \times n$ (or $3 \times n$) geometric Jacobian evaluated at the end-effector, restricted to that arm's own degrees of freedom ($n = 7$ for the Franka FR3, $n = 6$ for HEAL). Equation (37) is the regularised counterpart of the Moore–Penrose pseudoinverse solution to $J \cdot \dot{q} = e$; the $\lambda 2I$ term guarantees a bounded, invertible solution near kinematic singularities at a small cost to tracking accuracy, analogous to the Tikhonov-regularised least-squares estimate used in the plant identification of Eq. (19)–(20). Each iteration re-clips the resulting joint angles to their mechanical limits before the next update. IK is re-solved at 50 Hz, decoupled from the 500 Hz physics step, and each arm's previous solution is used as the warm-start *qinit* for the next iteration to preserve joint-space continuity between updates — the simulated counterpart of the 50 Hz control-cycle budget described in Section VI.E.

The two realizations share the same outer error-driven tilt-correction structure but differ in how that tilt is converted to actuation. The hardware plant of Section V commands a single physical joint offset directly from Eq. (17), augmented with predictive lookahead (Eq. 13), scenario-conditioned gain scheduling (Table I), online RLS/LMS plant identification (Eq. 19–22), and backlash compensation (Eq. 24–26). The simulated dual-manipulator plant instead treats the desired plate pose as an intermediate abstraction: a non-predictive PID (Eq. 30–31) produces the same class of tilt command, but that command must additionally be converted through quaternion composition (Eq. 33–35) and per-arm damped least-squares inverse kinematics (Eq. 36–37) before it can be realized as joint-space actuation, since no single physical axis corresponds directly to plate tilt. This confirms that the underlying error-driven tilt law of Section V is a property of the ball-and-plate abstraction itself, separable from the specific mechanism used to realize plate tilt, whether a single Dynamixel joint or a pair of cooperating redundant manipulators.



E. Simulation Results

The controller of Sections IX.B–IX.D was evaluated headlessly (no interactive viewer) against the real MuJoCo rigid-body physics at its native 500 Hz integration rate, using the actual, unmodified BallBalancer/IKSolver classes of `ball_balance_controller.py`. Table IV reports step-response statistics over 12 randomized initial ball offsets per axis, each drawn uniformly from $[0.08, 0.12]$ m.

A consistent axis asymmetry was found: the pitch axis (ϕ , controlling ball motion along plate-local y , realized through the Franka arm's position-and-orientation IK) settles in 3.43 s on average with 37.4% overshoot, while the roll axis (θ , controlling motion along plate-local x , realized through the HEAL arm's position-only IK) takes nearly twice as long to settle (6.56 s) with substantially larger and more variable overshoot (94.6% $\pm$ 41.9%). This is not predicted by the bi-axial PID law of Eq. (30)–(31), which is symmetric between the two axes by construction, and is therefore attributable to the asymmetric IK treatment between the two arms (Section IX.D): the Franka arm's orientation constraint couples plate pitch directly to a fully-determined wrist orientation, whereas the HEAL arm is solved for position only specifically to avoid fighting the rigid weld, leaving its axis more compliant and slower to settle under an identical PID output. Fig. 8 shows representative single-axis and off-diagonal step responses, and Fig. 9 shows the corresponding smoothed tilt commands saturating at the ± 0.07 rad limit during the initial transient of the more heavily excited roll axis.

This finding is a direct, reproducible consequence of realizing an idealized decoupled bi-axial control law (Eq. 30–31) through two mechanically and kinematically dissimilar manipulators, and represents exactly the kind of coupling effect the hardware-plant equations of Section V cannot expose, since that plant has no analogous mechanical asymmetry between its two tilt axes.

Fig. 10 shows a representative off-centre-target trial, where the pitch axis converges within the 9 s window while the roll axis is still slowly converging -- the same asymmetry visible in aggregate in Fig. 11, which plots the full per-trial spread underlying Table IV and shows the two axes' distributions do not overlap on settling time or overshoot across any of the 12 trials.

Fig. 8. MuJoCo dual-manipulator digital twin: single-axis step response (left, $x_0=0.12$ m) and off-diagonal step response (right, $(x_0,y_0)=(0.10,-0.10)$ m), plate-local ball coordinates.

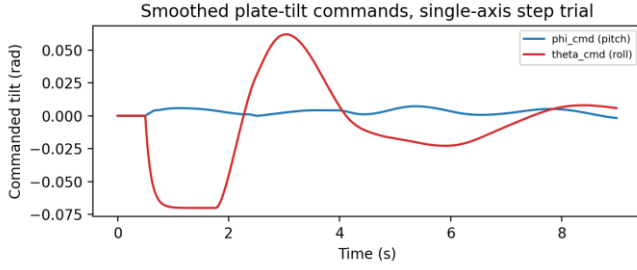


Fig. 9. Smoothed plate-tilt commands ϕ_cmd , θ_cmd over the single-axis step trial of Fig. 8, showing the roll command saturating at the ± 0.07 rad limit during the initial transient.

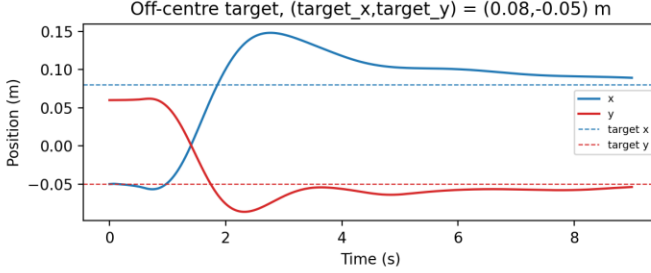


Fig. 10. Off-centre target trial, $(target_x, target_y) = (0.08, -0.05)$ m: the y-axis (pitch) settles near the target within the 9 s window while the x-axis (roll) is still slowly converging, consistent with the axis asymmetry of Table IV.

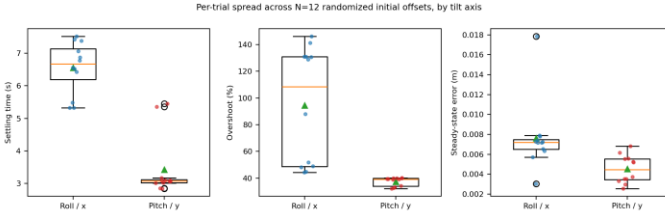


Fig. 11. Per-trial spread of settling time, overshoot, and steady-state error across the $N=12$ randomized-offset trials underlying Table IV, by tilt axis (individual trials overlaid as points; box shows quartiles, triangle shows the mean).

TABLE IV. MUJOCO DUAL-MANIPULATOR STEP-RESPONSE STATISTICS BY TILT AXIS, RANDOMIZED OFFSET IN $[0.08, 0.12]$ m, $N=12$ TRIALS PER AXIS

Tilt Axis	Settling Time (s)	Overshoot (%)	Steady-State Error (cm)
Pitch / y (Franka-orientation-coupled), $N=12$	3.43 ± 0.89	37.4 ± 3.1	0.45 ± 0.13
Roll / x (HEAL position-only IK), $N=12$	6.56 ± 0.76	94.6 ± 41.9	0.76 ± 0.33

X. CONCLUSION AND FUTURE WORK

This paper presented the perception, estimation, and control architecture of the OSRS Ball Balancer research module, developed as an experimental platform for investigating force/tactile-based robotic state perception as a complement to, and in limited cases a substitute for, vision. By combining predictive PD/PID control with forward error projection, scenario-aware gain scheduling, cubic polynomial force-to-position mapping, confidence-weighted vision-force sensor fusion, and online RLS/LMS parameter identification sustained by active persistent excitation, the platform achieves real-time stabilization of an under-actuated, open-loop-unstable physical interaction while estimating its own plant dynamics online. The four-phase development trajectory of Section VI documents a practical path from a minimal single-axis, vision-only proof of concept to a dual-axis hybrid vision-force estimator, and the modular event-driven software stack of Section III allows individual perception or control components to be replaced or extended independently. Quantitative software-in-the-loop evaluation (Section VII) and a MuJoCo-based dual-manipulator digital twin (Section IX) corroborate the control law with measured settling-time, overshoot, and steady-state-error statistics, while also surfacing concrete, previously undocumented implementation limitations — an active-exploration target disconnect and wall-clock-sensitive velocity estimation in the hardware path, and a pitch/roll asymmetry induced by asymmetric per-arm inverse kinematics in the simulated path — that future revisions of both controllers should address.

As discussed in Section VIII, the present system demonstrates contact-derived state estimation integrated into real-time closed-loop control; it does not demonstrate general tactile object recognition, and the phone-in-a-bag scenario motivating this work remains a future research direction rather than an experimental result. Future work is organized as a progression from the present system toward that longer-term goal.

Stage 1: Improve force-sensing resolution and calibration accuracy of the existing dual-load-cell channel. Stage 2: Replace or augment the load cells with distributed tactile sensing (e.g., force-sensitive resistors, capacitive or piezoresistive tactile arrays, or fingertip force/torque sensing). Stage 3: Collect tactile interaction signatures for a set of physically distinct objects. Stage 4: Extract features relating to contact location, force distribution, pressure pattern, shape, size, stiffness, and texture from these signatures. Stage 5: Develop tactile object classification using the extracted features. Stage 6: Combine tactile perception with active manipulation and exploration strategies. Stage 7: Perform vision-free object selection among multiple physically present objects. Stage 8: Demonstrate scenarios such as selecting a specified object from a cluttered container without visual input. Stage 9: Integrate the

resulting tactile-perception architecture into a humanoid robotic hand or arm.

Stages 1-2 extend directly from the hardware architecture already implemented; Stages 3-9 require sensing hardware, data collection, and classification methods beyond the scope of the present system and are identified as the natural continuation of this research.

ACKNOWLEDGMENT

The authors would like to acknowledge Prof. Dr. Harish Palanthandalam-Madapusi, Dean, Indian Institute of Technology Gandhinagar (IITGN), for his guidance and oversight during the internship under which this research was conducted. The authors also thank Dr. Rabinder Henry for facilitating the internship opportunity at IITGN.

REFERENCES

- [1] K. J. Åström and T. Hägglund, *Advanced PID Control*. Research Triangle Park, NC, USA: ISA, 2006.
- [2] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1999.
- [3] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," Univ. of North Carolina at Chapel Hill, TR 95-041, 2006.
- [4] D. Simon, *Optimal State Estimation: Kalman, H ∞ , and Nonlinear Approaches*. Hoboken, NJ, USA: Wiley, 2006.
- [5] R. J. Wai and J. D. Lee, "Fuzzy neural network control for a ball-and-plate system," *IEEE Trans. Ind. Electron.*, vol. 58, no. 12, pp. 5461–5473, Dec. 2011.
- [6] Z. Bingul and S. Karahan, "Fuzzy logic based nonlinear PID controller for a ball and plate system," in *Proc. Int. Conf. Robotics and Automation Sciences*, 2012.
- [7] Z. Zabinski, K. Åström, and B. Wittenmark, *Adaptive Control*, 2nd ed. Mineola, NY, USA: Dover Publications, 2008.
- [8] M. Bradley and R. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 3rd ed. Cham, Switzerland: Springer, 2023.
- [9] Intel Corp., "Intel RealSense D400 Series Product Family Datasheet," Rev. 010, 2023.
- [10] Robotis Inc., "Dynamixel X-Series e-Manual," [Online]. Available: <https://emanual.robotis.com>.
- [11] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Vilamoura, Portugal, 2012, pp. 5026–5033.
- [12] S. R. Buss, "Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods," Dept. Math., Univ. California San Diego, Tech. Rep., 2004.
- [13] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Boston, MA, USA: Pearson, 2018.
- [14] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009.
- [15] K. Shoemake, "Animating rotation with quaternion curves," in *Proc. ACM SIGGRAPH*, 1985, pp. 245–254.
- [16] F. Caccavale, P. Chiacchio, A. Marino, and L. Villani, "Six-DOF impedance control of dual-arm cooperative manipulators," *IEEE/ASME Trans. Mechatronics*, vol. 13, no. 5, pp. 576–586, Oct. 2008.
- [17] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE J. Robot. Autom.*, vol. 3, no. 1, pp. 43–53, Feb. 1987.
- [18] K. Zakka, Y. Tassa, et al., "MuJoCo Menagerie: A collection of high-quality models for the MuJoCo physics engine," GitHub repository, 2022. [Online]. Available: https://github.com/google-deepmind/mujoco_menagerie.
- [19] R. S. Dahiya, G. Metta, M. Valle, and G. Sandini, "Tactile sensing—From humans to humanoids," *IEEE Trans. Robot.*, vol. 26, no. 1, pp. 1–20, Feb. 2010.
- [20] S. J. Lederman and R. L. Klatzky, "Hand movements: A window into haptic object recognition," *Cogn. Psychol.*, vol. 19, no. 3, pp. 342–368, Jul. 1987.
- [21] H. Yousef, M. Boukallel, and K. Althoefer, "Tactile sensing for dexterous in-hand manipulation in robotics—A review," *Sens. Actuators A, Phys.*, vol. 167, no. 2, pp. 171–187, Jun. 2011.
- [22] R. S. Johansson and J. R. Flanagan, "Coding and use of tactile signals from the fingertips in object manipulation tasks," *Nat. Rev. Neurosci.*, vol. 10, no. 5, pp. 345–359, May 2009.
- [23] K. Hang, M. Li, J. A. Stork, Y. Bekiroglu, F. T. Pokorny, A. Billard, and D. Kragic, "Hierarchical fingertip space: A unified framework for grasp planning and in-hand grasp adaptation," *IEEE Trans. Robot.*, vol. 32, no. 4, pp. 960–972, Aug. 2016.